abyss

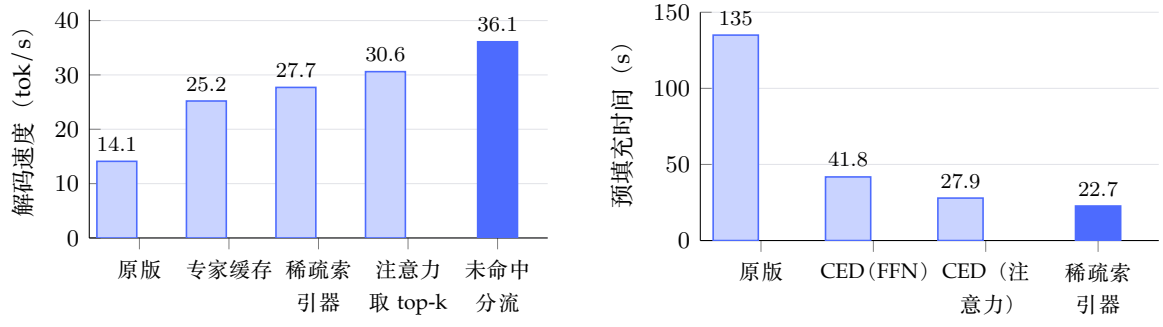
Abyss：在两张显卡上分层运行 DeepSeek-V4.1-Flash

Abyss

2026 年 10 月

摘要

长上下文智能体把推理的压力从算力推到了显存、内存和 PCIe 上。DeepSeek-V4.1-Flash 总参数 763B，原生权重约 474 GiB，超过一台双卡工作站（2 × RTX PRO 6000 Max-Q，显存合计 191 GiB；内存 192 GB）的全部容量。我们在 mx-llama.cpp 上实现了一个分层推理调度器 Abyss：编码器各层的专家常驻显存，解码器后 18 层的专家放在主机内存，由 GPU 上的专家缓存和 CPU 分担，Engram 大表留在 NVMe。围绕这款模型的编码器-解码器结构（CED）、压缩注意力和稀疏索引器，我们依次实现了 CED 感知预填充、Engram 预取、官方稀疏索引器、解码注意力的 top-k 收集、按热度准入的专家缓存未命中分流，以及预填充大批次的 CPU 分流。在保持原生 MXFP4 精度的前提下（每项改动对改动前的 KLD 都在数值底线以内），4.8 万上下文的单流解码从 14.1 tok/s 提升到 36.1 tok/s，12.8 万上下文为 36.2 tok/s。预填充的提升更大：4.8 万 token 提示词从 135 s（355 tok/s）降到 22.7 s（2,111 tok/s），快约 6 倍，其中 CED 感知预填充把解码器层的计算量从约 96 万个「token × 层」降到约 2.9 万个，贡献最大；12.8 万 token 的提示词只需 51.8 s（2,464 tok/s）；Engram 页缓存冷时，预取把冷读开销从约 39 s 降到约 2 s；多轮智能体会话里，之后每轮（1,200–1,600 token）的预填充从 5.7–7.2 s 降到 4.6–5.6 s。我们同时记录了评估方法（两档数值底线、对官方实现的单层对拍、逐 token 剖析），以及在真实 Codex 负载中发现并修复的缺陷。



(a) (b)

图 1 | (a) 4.8 万上下文的单流解码速度，从左到右依次叠加各项优化（第 4 节）。(b) 4.8 万 token 提示词的预填充时间，Engram 页缓存已热（第 5 节）；Engram 预取只影响冷缓存，不在图中。

目录

1 引言 **3**

2 硬件与模型 **3**

2.1 硬件 3

2.2 模型 4

2.3 权重格式与量化 5

2.4 方案 B 的布局 5

3 评估方法 **7**

3.1 数值底线 7

3.2 对官方实现的对拍 8

3.3 剖析 8

4 解码 **8**

4.1 GPU 专家缓存 8

4.2 解码注意力只取 top-k 行 9

4.3 按热度准入的未命中分流 9

5 预填充 **10**

5.1 CED 感知预填充 10

5.2 Engram 预取 10

5.3 补上官方稀疏索引器 11

5.4 大批次分流 11

6 服务与智能体负载 **12**

6.1 多轮会话的缓存复用 12

6.2 prompt 缓存的选择 12

6.3 真实负载暴露的缺陷 13

7 结论、局限与未来方向 **13**

A 补丁清单 **15**

B 复现 **15**

1 引言

智能体把大模型的典型请求从「短问短答」变成了「长上下文、多轮、带工具调用」。一个编程智能体每一轮都要在几万 token 的上下文后面追加回复、工具结果或追问，再让模型继续生成。这种负载有两个特点：一是上下文长，注意力和 KV 的开销随之增长；二是每一轮新增的内容不多，但首 token 之前的那段预填充决定了用户等待的时间。

DeepSeek-V4.1-Flash [1] 是为这类负载设计的。它有 552B 主干参数、196.6B 的 Engram 查表和约 14B 的 DSpark 草稿模型；40 层中前 20 层是因果编码器、后 20 层是解码器（CED），提示词的每个 token 只需走完编码器，生成的 token 才走完全部 40 层；注意力把 KV 按 2 倍或 1 倍压缩，再由稀疏索引器为每个查询挑出 512 个压缩位置；路由专家 384 个，每个 token 选 6 个，权重是原生 MXFP4。

问题在于容量。原生权重约 474 GiB（专家 268.9 GiB，Engram 189 GiB），而我们的机器只有两张 RTX PRO 6000 Max-Q（每张 95.6 GiB）和 192 GB 内存，合计约 383 GiB。公开的双卡方案 [3] 把专家量化到 3.0 bpw，速度很快（同样两张 RTX PRO 6000，单流解码 66–74 tok/s，4.6 万 token 提示词预填充约 1 万 tok/s），但相对原版的 KL 散度为 0.0545、top-1 一致率 92.9%，而且需要约 250 GiB 空闲内存，本机放不下。我们选择了另一条路：保留原生 MXFP4，把权重按热度分布在显存、内存和 NVMe 三层（方案 B），在 mx-llama.cpp [2] 上实现调度器 Abyss，再逐项找出瓶颈并优化。

本报告记录这一过程。主要结果见表 1：

- **解码**：GPU 专家缓存、解码注意力只取 top-k 行、按热度准入的未命中分流，使 4.8 万上下文的解码从 14.1 tok/s 提升到 36.1 tok/s，而且基本不再随上下文变长而变慢（12.8 万上下文为 36.2 tok/s）。
- **预填充**：CED 感知预填充让解码器层只计算后面还能看到的 token，加上 Engram 预取和官方稀疏索引器，4.8 万 token 提示词从 135 s 降到 22.7 s；大批次分流再把多轮会话每轮的预填充降低约 20%。
- **正确性**：所有改动都有对拍。我们区分了两档数值底线，并对稀疏索引器和解码注意力做了对官方 PyTorch 实现、对 float64 的逐层对拍。
- **真实负载**：在 Codex 的实际请求中发现了 prompt 缓存的选择问题和三处只在特定提示词长度下出现的崩溃，均已修复。

2 硬件与模型

2.1 硬件

表 2 是机器的配置和实测带宽。两点决定了后面的设计：内存是双通道、2DPC 运行在 3600 MT/s，纯读只有约 44 GB/s；两张卡各走 PCIe 5.0 x8，单卡主机到显存约 28.9 GB/s，但两卡同时从主机读合计只有 43.4 GB/s，被内存带宽卡住。所以无论专家由 CPU 计算还是上传到 GPU，每读一个专家都要占用同一份内存带宽。

表 1 | 各阶段的改动与效果。解码在 4.8 万上下文下测；预填充为 4.8 万 token 提示词、Engram 页缓存已热；KLD 是对改动前的 logits，「—」表示没有改变数值。

补丁	改动	效果	对拍
环境变量	预填充的 MXFP4 矩阵乘改用 8 bit 激活（W4A8）	预填充约慢 5%	0.044 → 0.012*
分支自带	GPU1 上的专家缓存（14 GB，约 780 槽位，LRU）	解码 14.1 → 25.2 tok/s	0.0005
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	预填充 135 → 41.8 s	0.00004
0004	Engram 整批预取	冷缓存 91.1 → 54.3 s	—
0005	解码器层的注意力也只算子集	41.8 → 27.9 s	0.00007
0006	补上官方稀疏索引器（top-512）	27.9 → 22.7 s；修正长上下文	见 §5.3
0008	解码注意力只取 top-k 行	解码 27.7 → 30.6 tok/s	0.003†
0009	未命中按热度一半上传、一半交给 CPU	30.6 → 36.1 tok/s	0.0003†
0014	预填充大批次分流	每轮 5.7–7.2 → 4.6–5.6 s	0.0005

* 预填充路径对解码路径的 KLD，不是对改动前。† server 上 temperature 0 生成时的 top-20 KL；0008 经过编码器层，属于较高的一档底线（§3.1）。

表 2 | 硬件与实测带宽（2026-10-20，STREAM 与 nvbandwidth）。

项目	配置与实测
GPU	2 × RTX PRO 6000 Blackwell Max-Q（300 W），每张 97,887 MiB GDDR7，SM120
互联	PCIe 5.0 x8 / x8，P2P 经 CPU，无 NVLink；卡间单向约 27.7 GB/s
CPU	AMD Ryzen 9 9950X，16 核 32 线程，单 NUMA
内存	4 × 48 GB DDR5，双通道 2DPC 运行在 3600 MT/s；纯读 44 GB/s，STREAM Copy 42 / Triad 32 GB/s
主机 → 显存	单卡 28.9 GB/s；两卡同时合计 43.4 GB/s
存储	Samsung 9100 PRO 4TB（PCIe 5.0）、990 PRO 4TB（PCIe 4.0）

2.2 模型

图 2 是 DeepSeek-V4.1-Flash 的逐层结构和方案 B 在本机的布局，维度取自 GGUF 元数据。与本报告相关的结构特点有四个：

- **编码器-解码器（CED）**。第 0–19 层是因果编码器，第 20 层的压缩器生成所有解码器层共享的全局 KV。解码器层只通过各自 128 个 token 的滑动窗口读自己的隐状态，因此提示词前面的大部分 token 不需要走完解码器。
- **压缩注意力**。第 0–1 层只有滑动窗口；第 2–19 层把每 2 个 token 压成一行，第 20–39 层每个 token 一行。压缩 KV 只在第 2、8、14、20 层写入，其后的层复用。每个查询有 64 个头，键值只有 1 个 512 维的头（K = V）。
- **稀疏索引器**。第 2、8、14、20、24、28、32、36 层用 32 个头、128 维、FP4 的查询和键给压缩位置打分，选出 top-512；第 20 层还按每 8 行一块挑出 2,048 个候选块，供第

24–36 层重选。其余层沿用最近索引源的结果。

- **专家与 Engram**。每层 384 个路由专家选 6 个，外加 1 个共享专家；每个 MXFP4 专家约 18.8 MB，每层 7.2 GB。第 1、14 层按最近 2–4 个 token 的 n-gram 哈希查两张约 3.84 亿行的大表。

2.3 权重格式与量化

本报告的优化都没有对模型再做量化：改动的只是权重放在哪一层，以及哪些 token、哪些行不必计算，每项改动都和改动前对拍（§3.1）。不过我们运行的 GGUF [2] 本身和官方 checkpoint 不完全相同。表 3 是逐个统计 12 个分片的结果。

表 3 | GGUF 中各类权重的格式，与官方 checkpoint 对比。

权重	官方	GGUF	大小	精度变化
路由专家	FP4（E2M1，每 32 个一个缩放）	MXFP4	275.7 GiB	无，同一种格式
注意力等稠密矩阵	FP8	Q8_0	5.5 GiB	很小，都是 8 bit，只是缩放方式不同
共享专家	FP8*	Q8_0	1.5 GiB	同上
Engram 表	**FP8，约 189 GiB**	**MXFP4**	**97.3 GiB**	**从 8 bit 降到 4 bit**
嵌入、输出、路由门、压缩器、索引器投影	—	BF16	2.8 GiB	无
各种 norm、偏置	—	F32	0.2 GiB	无

* 按官方 `inference/model.py` 推断：只有路由专家用 FP4，其余线性层走 FP8 矩阵乘；共享专家没有单独核对。
—：未单独核对，GGUF 中是 16 位或 32 位浮点。

真正降了精度的只有 Engram 表。转换 GGUF 时它从 FP8 压到了 MXFP4，体积减半，也更容易留在页缓存里。这是转换者的选择，我们直接继承；本报告的对拍都是同一个 GGUF 改动前后的比较，这一步对质量的影响没有测过，要测需要用官方 FP8 的表做参照。

计算时还有两处量化。GPU 和 CPU 上的矩阵乘都先把激活按 32 个一组量化成 int8 再乘，即 W4A8；官方参考实现用 FP8 激活，同为 8 bit。fork 在 Blackwell 上的预填充默认是 W4A4（激活只有 4 bit），我们改回了 8 bit（§3.1）。稀疏索引器的查询和键用 FP4，这是官方的设计，我们照官方实现做。KV cache 用默认的 F16，没有量化。

作为对比，方案 A 要把路由专家从约 4.25 bpw 再压到 3.0 bpw，相对原版的 KL 散度为 0.0545 [3]；方案 B 不做这一步，这正是选它的原因。

2.4 方案 B 的布局

方案 B 用层切分（`-sm layer -ts 12/29`）：GPU0 放第 0–11 层整层；GPU1 放第 12–39 层的稠密部分和第 12–21 层的专家；第 22–39 层的专家（121 GiB）放在锁页主机内存；Engram 表留在 NVMe，经页缓存读取。编码器的专家全部在显存，所以只要提示词不走解码器层，预填充就基本不碰主机内存。表 4 列出各层的内容。后来加入的 GPU1 专家缓存占 14 GB，大

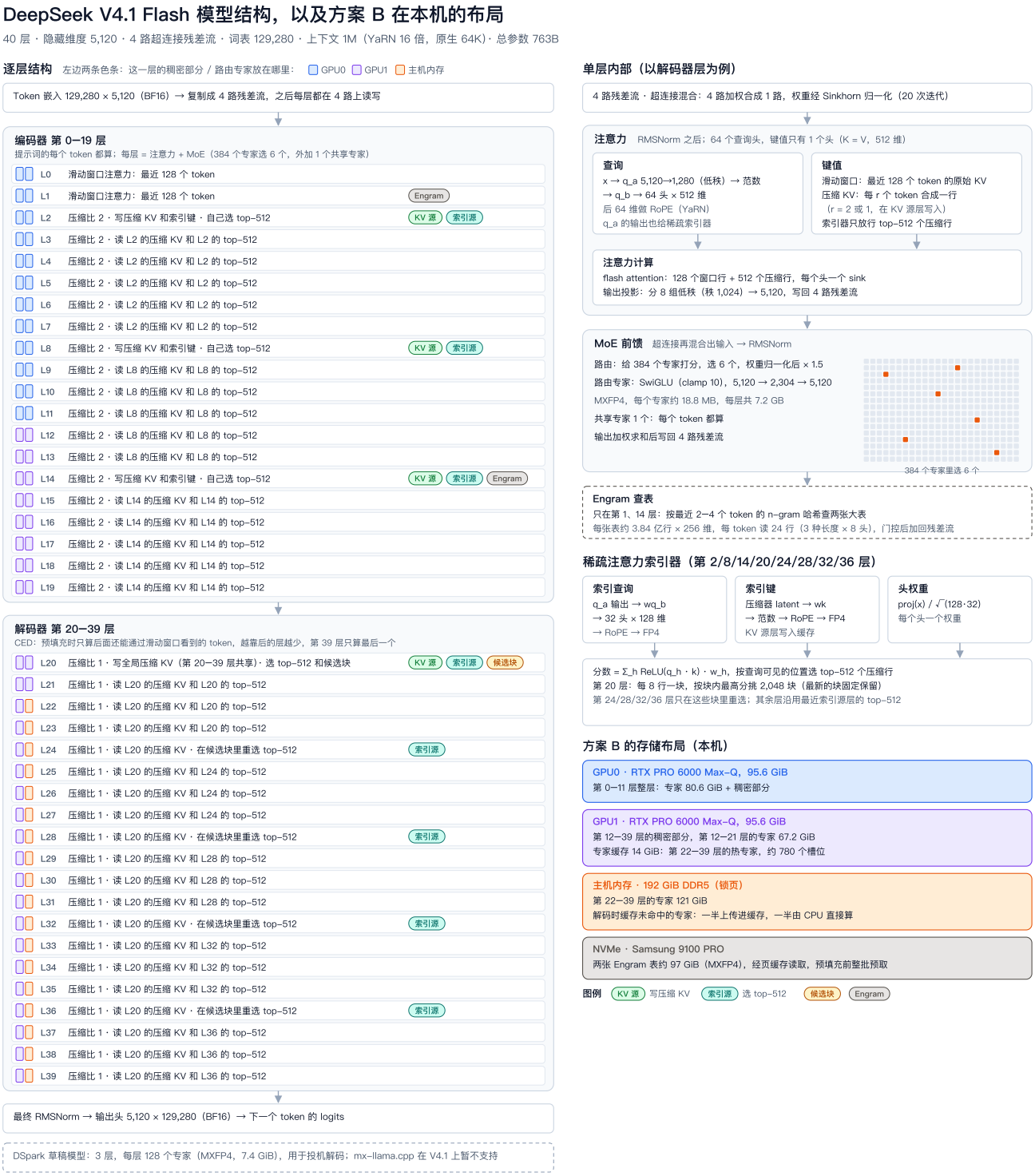


图 2 | DeepSeek-V4.1-Flash 的逐层结构，以及方案 B 在本机的布局。左侧每行一层，色条表示该层稠密部分和路由专家所在的位置；右侧是单层内部、稀疏索引器和存储布局。

批次分流让 GPU0 的计算缓冲区多占约 1.9 GB；12.8 万上下文时，两张卡分别只剩约 3.6 GB 和 1.2 GB。

表 4 | 方案 B 的存储布局（最终配置）。

位置	内容	容量
GPU0	第 0–11 层整层（专家 80.6 GiB + 稠密部分）	95.6 GiB
GPU1	第 12–39 层稠密部分，第 12–21 层专家 67.2 GiB，专家缓存 14 GB	95.6 GiB
主机内存	第 22–39 层专家 121 GiB（CUDA_Host 锁页）	192 GB
NVMe	两张 Engram 表（MXFP4，约 97 GiB）	4 TB

3 评估方法

3.1 数值底线

我们用 llama.cpp 的 KL 散度对拍：先存一份基准 logits，再比较改动后的 logits，同一配置重跑结果完全相同（KLD 为 0）。重要的发现是数值底线分两档（表 5）：

- 改动只落在解码器层（第 20–39 层）时，KLD 约 0.0005。例如把第 22–39 层的专家从 CPU 改到 GPU 上计算。
- 改动经过编码器层（第 0–19 层）时，0.005–0.01 属正常。例如只把第 5 层的专家改到 CPU 计算就是 0.0064；换内核、关融合、换 ubatch 也在这个量级。

原因在 CED：编码器的输出决定了所有位置共享的全局 KV，那里的舍入差别会扩散到之后所有位置；解码器层的差别只影响当前 token。判断一项改动是否正确，必须先确定它落在哪一档。

同一套方法一开始就查出了一个精度问题：mx-llama.cpp 在 Blackwell 上对 MXFP4 的预填充默认用 W4A4（激活量化到 4 bit），而官方参考实现是 W4A8。预填充路径对解码路径的 KLD 因此高达 0.044、top-1 一致率 91%；改为 8 bit 激活后降到 0.012、95%，与数值底线相当，预填充只慢约 5%。

表 5 | 数值底线与 W4A4 问题（2026-10-21）。

比较	改动范围	KLD
同一配置重跑	—	0
第 22–39 层专家从 CPU 改到 GPU	只有解码器层	0.0005
第 5 层专家从 GPU 改到 CPU	经过编码器层	0.0064
预填充 W4A4 对解码路径	全部层	0.044
预填充 W4A8 对解码路径	全部层	0.012

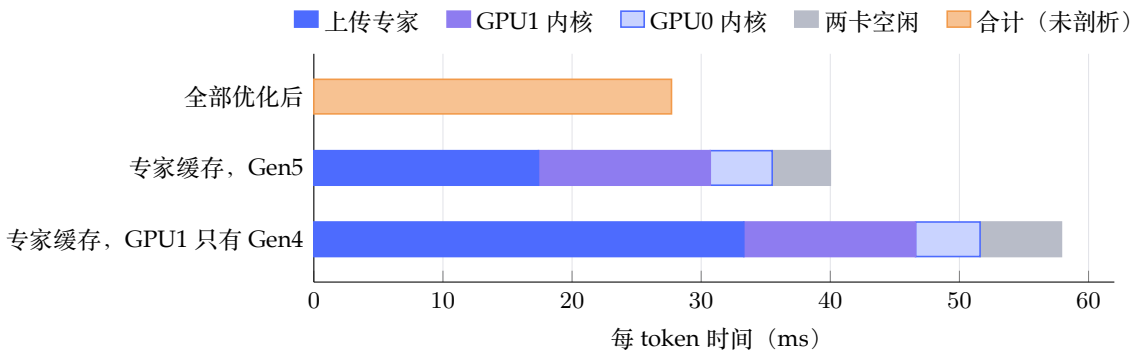


图 3 | 4.8 万上下文解码的每 token 时间。中间一行是开启专家缓存后的剖析；下面一行是一次开机后 GPU1 的链路只训练到 Gen4 时的剖析，上传慢了一倍（§6.3）；上面一行是本报告全部优化后的总时间（36.1 tok/s）。

3.2 对官方实现的对拍

KLD 只能说明最终输出变了多少，不能说明是哪一步造成的。对实现复杂的部分，我们写了独立的参照实现（oracle）逐层对拍：用 `llama-abyss-dump` 导出 fork 计算图里指定的张量，再用 PyTorch 按官方 `inference/model.py` 从同样的输入重算。解码步的注意力还用 float64 重算，单独量出内核的舍入误差。具体结果见 §5.3 和 §4.2。

3.3 剖析

性能问题一律先剖析再动手。我们用 nsys 抓取一段预填充或一段解码，按设备统计内核、拷贝和空闲时间，再按 token 平均（`tools/eval/nsys_breakdown.py`、`nsys_decode.py`）。多轮智能体用一个模拟 Codex 的脚本测量：首轮带 4.8 万 token 的文档和两个工具，之后每轮回填约 1,500 token 的工具结果或追问，逐轮记录缓存复用和预填充时间（`agent-turns.sh`）。

4 解码

方案 B 的原版解码是 16.5 tok/s（空上下文），4 万上下文时约 14.6 tok/s。瓶颈是 CPU：解码器后 18 层每个 token 要读 108 个专家、约 2.0 GB，而内存带宽只有 44 GB/s。

4.1 GPU 专家缓存

mx-llama.cpp 自带一个实验性的专家缓存：在 GPU1 上划出一组槽位（bank），每层一张「专家号 → 槽位号」的映射，命中的专家直接在 GPU 上算，未命中的同步上传并按 LRU 淘汰。我们给它 14 GB（约 780 个槽位，占第 22–39 层专家的 11%）。真实文本的专家局部性很强，LRU 命中率约 75%；4.8 万上下文的解码从 14.1 提升到 25.2 tok/s，短上下文约 29–30 tok/s，KLD 0.0005。

开启缓存后，我们在 PCIe Gen5 下剖析了 4.8 万上下文的解码（图 3）：每 token 40.1 ms，其中 GPU1 上传未命中专家 17.4 ms（约 26.5 个、474 MiB），GPU1 内核 13.3 ms，GPU0 内核 4.8 ms，两卡都空闲 4.5 ms，几部分基本串行。上传是最大的一块。

4.2 解码注意力只取 top-k 行

剖析还显示 GPU1 内核里有约 3 ms 花在 `concat_cont` 上，而且随上下文增长：每层每个 token 都把滑动窗口和整个压缩 KV 拼成一个张量（4.8 万上下文时每次约 40 μs），再由 flash attention 按 top-512 掩码挑行。单个查询时这是浪费的：它只会看到 512 个压缩行。我们改为先用 `get_rows` 取出这 512 行和它们的掩码，再和滑动窗口拼接，补齐到 256 的倍数后走普通 flash attention。

4.8 万上下文的解码从 27.7 提升到 30.6 tok/s，12.8 万上下文从 23.9 提升到 30.9 tok/s，解码速度基本不再随上下文变长而下降。server 上 temperature 0 生成的 top-20 KL 约 0.003，高于解码器层那一档底线。我们用 float64 对拍确认了原因：给定相同输入时，两条路径看到的是同一组 640 行，和 float64 的误差都约 5e-4（F16 舍入）；第 20 层的差别来自上游——第 2–19 层约 5e-4 的舍入差别经 MoE 在分数接近的专家之间选择而被放大，这正是「经过编码器层」那一档的机制。

4.3 按热度准入的未命中分流

原设计是把未命中全部交给 CPU 计算，只按静态热表换入。剖析后发现这行不通：LRU 的高命中率靠的是每个未命中都换入，不换入时命中率会掉到静态热表的水平（每 token 约 47 次未命中），CPU 计算反而比上传还慢；CPU 算完再把同一个专家上传，又要从内存读两遍。

我们改为按热度准入。每层由一个 CPU 算子规划：命中的照常读缓存；未命中的按衰减热度（每层每步乘 0.99）挑出一定比例，立即异步上传进 LRU 槽位；其余由 CPU 用主机内存里的原始专家计算，GPU 那边读 bank 末尾的全零槽位，down 之后两边相加。计算图里 CPU 分支先于 GPU 分支展开，规划算子在 CPU split 开头发出上传，CPU 计算其余未命中时 DMA 正在进行，每个专家只从内存读一遍。

表 6 是上传比例的扫描。只上传一半，命中率不降：按热度挑出来的才进缓存，冷门专家不会把缓存冲掉。最终取 0.5：4.8 万上下文 36.1 tok/s，12.8 万上下文 36.2 tok/s，KL 约 2e-4，低于解码器层底线。

表 6 | 未命中上传比例的扫描（4.8 万上下文，已开启 top-k 收集，复用提示词缓存后生成 256 token）。

上传比例	解码（tok/s）	命中率	对全部上传的 top-20 KL
全部上传（原缓存）	30.6	75.5%	—
1.0（新路径）	30.2	75.5%	0（逐位一致）
0.5	36.1	75.9%	2.7e-4
0.4	35.7	75.1%	2.2e-4
0.3	36.4	74.5%	1.7e-4
0.2	36.0	73.1%	2.2e-4
0（全部交给 CPU）	16.1	0%	1.8e-4

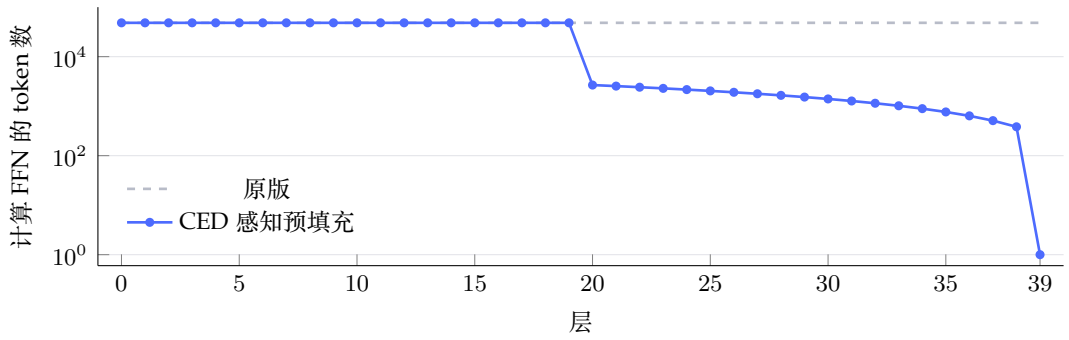


图 4 | 4.8 万 token 提示词每层计算 FFN 的 token 数（按实现中的规划规则计算，含约 256 个位置的余量）。编码器层全部计算；解码器层只计算后面还能通过滑动窗口看到的 token。

5 预填充

原版对提示词也跑完 40 层：每个 2,048 token 的 ubatch 都要把主机内存里 121 GiB 的解码器专家读一遍。4.8 万 token 的提示词因此要 135 s（约 355 tok/s）。

5.1 CED 感知预填充

官方参考实现对提示词也跑满 40 层，CED 体现在 KV 的共享方式上：解码器层只有滑动窗口用到各层自己的隐状态。因此可以精确地裁剪：第 0–19 层全部 token 都算；第 39 层只算最后一个 token；第 L 层（20–38）只需算最后 $128 + 127 \times (38 - L)$ 个 token——每往前一层，滑动窗口让依赖往前延伸 127 个 token。实现中再多留约 256 个位置的余量。这样解码器层合计约 2.9 万个「token × 层」，与提示词长度无关；4.8 万 token 的提示词在原版中是 $20 \times 48{,}240 \approx 96$ 万个（图 4）。

实现分两步。第一步（`0003`）只跳过 FFN，注意力仍对所有 token 计算，KV 写入完全不变；被跳过的 token 之后各层的隐状态不正确，但只会进入没有后续 token 读取的滑动窗口。每次 `llama_decode` 按自己的末尾规划，所以 server 用 `-b 65536` 让整段提示词进一次调用。4.8 万 token 的预填充从 135 s 降到 41.8 s，8k/16k 上下文对拍 KLD 0.00006 / 0.00004，top-1 一致率 99.8%。

第二步（`0005`）先剖析：此时 41.8 s 里第 20–39 层的注意力占 19.9 s。我们让这些层仍为所有 token 写 KV，但只对 CED 子集取出查询和掩码行计算注意力。预填充降到 27.9 s（1,715 tok/s），KLD 0.00004–0.00007，temperature 0 的输出和第一步一字不差。

5.2 Engram 预取

Engram 表以 mmap 映射且设了随机访问提示，每次缺页都同步读一个 4 KiB 页，2,048 个 token 约需 2.4 s，瓶颈在缺页延迟而不在 SSD 带宽。由于行号只取决于输入 token，我们在 `llama_decode` 拿到整批后立即算出全部行号，由 8 个后台线程按地址排序后用 `MADV_WILLNEED` 预读。页缓存冷时，4.8 万 token 的预填充从 91.1 s 降到 54.3 s，冷读开销从约 39 s 降到约 2 s；热缓存时不变，结果与不预取完全一致。冷读已基本藏在计算后面，io_uring 和按行缓存等后续步骤暂缓。

5.3 补上官方稀疏索引器

fork 原本对 V4.1 的压缩 KV 做稠密注意力。压缩行在 512 以内时与官方的 top-512 选择等价，更长时就不同了：压缩比 1 的层超过约 512 个 token、压缩比 2 的层超过约 1,024 个 token 之后。智能体的上下文几乎都在这个范围里，所以这首先是正确性问题。

我们按官方 `inference/model.py` 补上了完整的索引器：KV 源层写 FP4 索引键（每 32 个一块，缩放 $2^{\lceil \log_2(\text{amax}/6) \rceil}$），索引源层计算 $\sum_h \text{ReLU}(q_h \cdot k)\, w_h$ 并选 top-512，第 20 层挑候选块，其余层沿用最近索引源的结果；flash attention 只看选中的位置。512 token 以内与稠密版 KLD 为 0；8k/16k/32k 上下文时，稀疏版的 PPL 比稠密版低 0.9% / 3.1% / 0.23%（表 7）。由于注意力只看 640 行，4.8 万 token 的预填充也从 27.9 s 降到 22.7 s，12.8 万 token 为 51.8 s（2,464 tok/s）。

表 7 | 稀疏索引器对稠密注意力（perplexity，每档 4 段）。

上下文	PPL（稠密）	PPL（稀疏）	变化	稀疏对稠密 KLD
8k	6.023	5.972	−0.9%	0.057
16k	3.009	2.916	−3.1%	0.094
32k	1.0027	1.0004	−0.23%	0.012

为确认实现与官方一致，我们导出第 2、20、24 层的索引器输入输出，用三种算术重算 top-512 并比较重合率（表 8）：全程 float32；fork 的算术（Q8_0 矩阵乘把激活量化成 int8）；官方的算术（bf16 激活，量化成 e4m3 后做 FP8 GEMM）。fork 与自己的算术重合 99.98% 以上，说明 RoPE/YaRN、FP4、权重缩放、可见性和 top-k 每一步都对；fork 离 float32 的距离（98.7–99.2%）比官方算法自己离 float32 的距离（97.1–98.7%）还近。索引键 99.99% 以上逐位一致，不一致的都是恰好落在 FP4 中点上的单个元素。

表 8 | top-512 集合的平均重合率（4,027 token 提示词）。

层	fork 对 int8 参照	fork 对 float32	官方算术对 float32	fork 对官方算术
2	99.99%	99.16%	98.67%	98.63%
20	99.98%	98.67%	97.14%	97.11%
24	99.98%	98.85%	97.40%	97.36%

5.4 大批次分流

多轮会话里每轮新增的 token 不多，但只要批次超过 32 个 token，第 22–39 层几乎全部 384 个专家都会被用到，调度器把这些大批次的专家矩阵乘卸载到 GPU0，每层整批上传。一轮 1,610 token 的预填充（7.0 s）中，GPU0 上传 145 GiB 用了 5.4 s，而 CPU、GPU1 的链路和专家缓存都闲着。这部分的下限是把这些专家从内存读一遍，约 3 s。

我们把未命中分流的思路推广到大批次：每层一个 CPU 算子统计每个专家的 token 数，按从少到多把专家交给 CPU，直到 CPU 的预计时间追上其余专家的上传时间；其余照常上

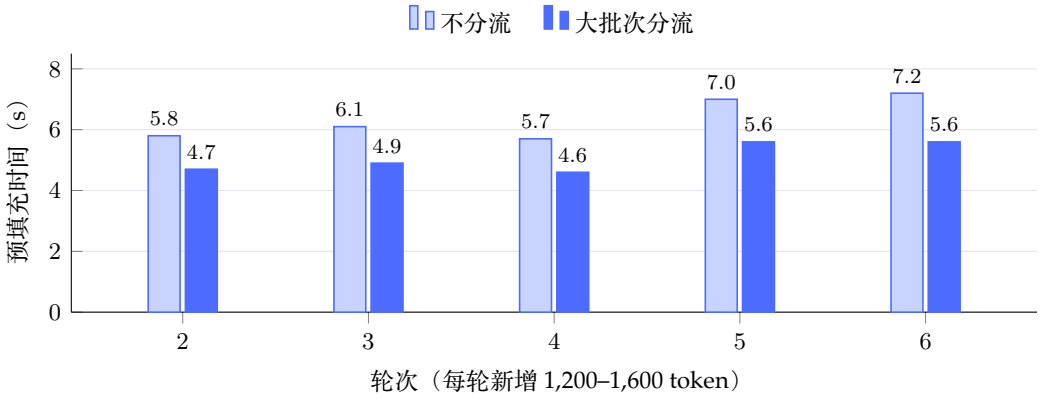


图 5 | 多轮智能体会话中每轮的预填充时间（首轮为 4.8 万 token 的文档，之后每轮回填工具结果或追问）。

传到 GPU0。为此给 CUDA 的 MMQ 补上了 id = −1 的处理（跳过并把输出清零），上传回调直接用规划算子在主机上的结果，不必从 GPU 读回 id。

第一版的 CPU 分支处理整块张量（全部 token × 6 个专家），而 CPU 实际只算约 5% 的行，量化、SwiGLU、清零和 252 MB 输出的回传把收益全吃掉了，比不分流还慢。改为紧凑批次后——只取出交给 CPU 的「token × 专家」（最多 1,024 行），在 GPU1 上取出这些 token，CPU 上按一行一个专家计算，`set_rows` 写回后与 GPU0 的结果相加——2,048 token 一批的预填充从 4.88 s 降到 4.24 s，KLD 0.00046。多轮会话之后每轮的预填充从 5.7–7.2 s 降到 4.6–5.6 s，首轮从 30.6 s 降到 27.9 s（图 5）。

这一步离下限还有约 30 ms/层：GPU0 的上传暂存区在 gate、up、down 三份权重之间复用，每段之间都要同步，CPU 要等 gate 那批传完才能开始；CPU 计算和 DMA 又共享 44 GB/s 的内存带宽。合并的几步要固定在 GPU0，否则 GPU1 的计算缓冲区会大 3.7 GB，12.8 万上下文时放不下。实现中还发现并修复了 fork 的一处问题（`0013`）：`ggml_backend_sched_reserve_size` 在切分计算图之前重置调度器，清掉了构建计算图时手动指定的后端，按错误的切分估算计算缓冲区。

6 服务与智能体负载

6.1 多轮会话的缓存复用

V4.1 的缓存不能回退到滑动窗口中间，只能退回检查点。server 在每条用户消息的开头、最后一条用户消息开头和提示词末尾附近创建检查点。模拟会话中，从第 3 轮起每轮只重算上一轮生成的回复加 4 个 token：回复经模板重新渲染后与生成时的 token 不完全一样，server 退回到「Assistant」前缀处的检查点。

6.2 prompt 缓存的选择

在 Codex 的 6 个真实请求中（预填充合计 76 s，生成 129 s），我们发现 server 只有一个槽位，Codex 的辅助请求（与主对话共享约 4,500 token 的系统提示和工具定义，只问一句）会把主

对话挤出槽位。server 已把主对话存进内存里的 prompt 缓存，但主对话回来时，槽位里的辅助请求与它仍有 94% 的前缀相同，而 server 只在保留比例低于一半时才去缓存里找，并且按保留比例挑选，长的条目反而落选。

我们改为按「能直接用上的 token 数」比较槽位和缓存条目——对检查点受限的缓存，就是不晚于分叉点的最近检查点的位置——多出 256 个以上就换入；只用到一小部分的条目复制一份，原条目留在缓存里。另一个问题是辅助请求自己也从头算：系统提示末尾有检查点，但 server 要求检查点比分叉点早至少 128 个 token。这是为保存滑动窗口的检查点定的规则；V4.1 的检查点是那个位置的完整状态（`pos_min == pos_max`），在分叉点之前就能接着算。修复后，同一场景里辅助请求从 8.4 s 降到 0.3 s（只重算 13 个 token），主对话的下一轮从内存缓存换回，只重算 216 个 token、2.2 s。

6.3 真实负载暴露的缺陷

- **未分配的输入张量。**CED 让某些层在某个 ubatch 里没有 token，依赖它们的计算被整段丢掉，预先建好的输入张量就不会分配，往里写数据时断言失败。这在 CED 行映射（提示词约 2,500–16,000 token）和候选块输入（约 1.7 万 token 以上、长度除以 2,048 余约 114–622）上各出现一次，都只在特定长度下触发，测速用的 4.8 万和 12.8 万 token 恰好都没碰到，由 Codex 的请求暴露。
- **全零槽位重复。**未命中分流最初只有一个全零槽位。9–32 token 的批次里，同一 token 的多个 CPU 未命中指向同一个槽位，CUDA MMQ 的重排每个专家每个 token 只记一行，漏掉的行号未初始化，量化内核越界。改为每个 token 的第 k 个专家用第 k 个全零槽位。
- **PCIe 降速。**有一次开机后 GPU1 的链路只训练到 Gen4，主机到 GPU1 只有 14.4 GB/s，未命中上传慢了一倍，解码从约 25 掉到 17 tok/s（图 3）；重启后恢复。测速脚本现在会先检查链路。

7 结论、局限与未来方向

在一台双卡工作站上，Abyss 以原生 MXFP4 精度运行 DeepSeek-V4.1-Flash：单流解码约 36 tok/s，且几乎不随上下文变长而变慢；4.8 万 token 提示词的预填充约 23 s，多轮会话每轮约 5 s。收益主要来自三类工作：顺着模型结构减少计算（CED 感知预填充、稀疏索引器、解码注意力只取 top-k），按热度安排数据所在的层级（专家缓存与准入），以及让 CPU、DMA 和 GPU 同时工作（未命中分流、大批次分流）。每一项都先剖析、后实现，并用分档的数值底线和对官方实现的对拍确认正确性。

局限。 解码器专家的读取最终受内存带宽限制：无论 CPU 计算还是上传，每个专家都要从 44 GB/s 的双通道内存读一遍，预填充的大批次和解码的未命中都已接近这个下限。显存余量很小（12.8 万上下文时两卡分别约 3.6 GB 和 1.2 GB），更大的缓存或更长的上下文要以缩

小专家缓存为代价。GGUF 中的 Engram 表从官方的 FP8 降到了 MXFP4（§2.3），这对质量的影响还没有测过。DSpark 投机解码在 V4.1 上尚不支持，这部分收益还没有拿到。

未来方向。

- 让超过 2,048 token 的提示词末段只上传一遍解码器专家（目前每个 ubatch 各传一遍），预计新提示词的预填充还能省数秒。
- 在 BIOS 中尝试把内存提到 5200 MT/s，带宽约提高 1.4 倍，所有受内存带宽限制的部分都会受益。
- 补上 V4.1 的 DSpark 投机解码。diffbot 在智能体编程会话中开启后解码约为单流的两倍；但在分层布局下，一次验证多个 token 会选中更多不同的专家，要多读的专家可能抵消一部分收益，需要实测。
- 与方案 A（3.0 bpw 专家、vLLM 双卡张量并行，预计 55–75 tok/s）做一次同机对比，量化速度与精度的取舍。

参考资料

[1] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[2] mxxm-t. mx-llama.cpp 与 DeepSeek-V4.1-Flash 的 MXFP4 GGUF. https://huggingface.co/mxxm-t/DeepSeek-V4.1-Flash-GGUF

[3] diffbot. DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000. https://huggingface.co/diffbot/DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000

[4] Strata：三层分工的本地 MoE 推理. https://github.com/Niko1221/Strata

A 补丁清单

Abyss 的全部改动以补丁形式保存在仓库的 `mx-llama.cpp/` 目录，基于 mx-llama.cpp 的提交 `c4d12e2`，按序号依次应用。

补丁	作用	开关
0001	server 的专家命中统计	`ABYSS_EXPERT_STATS`
0002	V4.1 的工具调用标签（DSML，标签名带前导空格）	—
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	`ABYSS_CED_PREFILL=1`
0004	Engram 整批预取	`ABYSS_ENGRAM_PREFETCH=8`
0005	CED 感知预填充：注意力也只算子集	同 0003
0006	V4.1 稀疏注意力索引器	`ABYSS_DSV41_INDEXER=1`
0007	`llama-abyss-dump`：导出计算图张量用于对拍	—
0008	单 token 解码时注意力只取 top-k 行	`ABYSS_DSV41_DECODE_GATHER=1`
0009	专家缓存未命中按热度分给上传和 CPU	`ABYSS_MOE_CPU_MISS=1`
0010	修 CED 行映射未分配；未命中分流的全零槽位重复	—
0011	修候选块输入未分配	—
0012	prompt 缓存按能直接用上的 token 数挑选	—
0013	修调度器估算显存时清掉手动指定的后端	—
0014	预填充大批次分流到 CPU	`ABYSS_MOE_BIG_SPLIT=1`

B 复现

生产配置的启动命令（需打过全部补丁的编译；还必须设 `GGML_CUDA_Q8_1_CACHE=0` 和 `GGML_CUDA_MMQ_PREC=q8`）：

```
GGML_CUDA_Q8_1_CACHE=0 GGML_CUDA_MMQ_PREC=q8 \
ABYSS_CED_PREFILL=1 ABYSS_ENGRAM_PREFETCH=8 ABYSS_DSV41_INDEXER=1 \
ABYSS_DSV41_DECODE_GATHER=1 ABYSS_MOE_CPU_MISS=1 ABYSS_MOE_BIG_SPLIT=1 \
llama-server -m DeepSeek-V4.1-Flash-MXFP4-00001-of-00012.gguf \
  -ngl 99 -sm layer -ts 12/29 -fa 1 -lm dio -c 131072 -b 65536 -ub 2048 \
  -ot "blk\.(2[2-9]|3[0-9])\.ffn_(gate|up|down)_exps\.=CPU" \
  -t 16 -np 1 --moe-cache-mib 14000 \
  --jinja --chat-template-file deepseek-v4.1.jinja
```

评估脚本在仓库的 `tools/eval/`，复制到机器上运行，运行前要先停 server：

脚本	用途
kl.sh	存基准 logits，比较 KLD
bench.sh	解码与预填充测速
prefill-warm.sh	server 热缓存预填充
profile-prefill.sh、profile-decode.sh	nsys 剖析预填充、解码
dump-indexer.sh、indexer_oracle.py	稀疏索引器的单层对拍
decode-gather.sh、decode_attn_oracle.py	解码注意力的 float64 对拍
engram-cold.sh、engram-share.sh	Engram 冷读开销与占比
m2-cpu-miss.sh	未命中分流的上传比例扫描
agent-turns.sh	多轮智能体会话